D.Y. PATIL COLLEGE OF ENGINEERING & TECHNOLOGY,

KOLHAPUR

(An Autonomous Institute)

DEPARTMENT OF CSE (DATA SCIENCE)

A

Synopsis Report

on

"RepoSensee-AI-Powered Dev Collaboration Platform"

Submitted by

Name	Roll No.
Sneha Suresh Abdar	109
Madhura Sandip Patil	110
Saksham Madhukar Wayadande	111
Priyanka Pramod Jadhav	112

Under the guidance of

Prof. Tejashri Deokar

Final Year B. Tech. CSE (Data Science)

Academic Year 2026-27

INDEX

Sr. No. Topic Page No.

1. Abstract 3
2. Introduction 4
3. Literature Survey 6
4. Motivation 9
5. Problem Definition 10
    a. Problem Statement 10
    b. Objectives 10
    c. Proposed Architecture 11
    d. Experimental Setup 13
6. References 15
7. Approval Statement 17
8. Plagiarism Report 18

ABSTRACT

Modern software teams rely on Git-based platforms to manage code, yet a great deal of what happens around the core development cycle is still done by hand. Reviewing pull requests, spotting bugs early, writing documentation, checking for security issues, and keeping a project's own coding conventions intact are activities that mature Git platforms leave almost entirely to human effort. Teams also have no automatic way of learning the naming and structural patterns that are unique to their own codebase — the linters available today enforce generic, one-size-fits-all rules instead of conventions that grow organically inside a project, and this gap leads to naming drift and inconsistent file structures as a repository grows and contributors come and go.

This synopsis presents the AI-Powered Dev Collaboration Platform, a Git-based development environment built with artificial intelligence woven into its core workflow rather than bolted on as an add-on. Beyond the usual repository management, branching, pull requests, and issue tracking, the platform brings together seven AI-driven modules: AI Code Review, AI Bug Detection, an AI Documentation Generator, an AI Project Assistant, an AI Security Scanner, Smart Analytics, and a newly proposed AI Consistency & Convention Analyzer. The last of these studies a repository's existing naming habits, groups files with similar structure into peer groups so comparisons stay fair, and flags deviations from the learned pattern along with a confidence score and supporting evidence, instead of applying blanket rules that ignore context.

By combining version control, team collaboration, and AI-assisted quality, security, documentation, and consistency checks inside one platform, the system is expected to cut down repetitive manual work, catch bugs and vulnerabilities sooner, keep documentation aligned with the actual code, and slow the naming and architectural drift that tends to creep in as a repository scales. It is designed for individual developers and larger engineering teams alike, and rests on a modern, cloud-native web architecture that draws on established research in code review automation, vulnerability prediction, documentation generation, convention learning, and code-smell detection.

INTRODUCTION

Software development has changed a great deal over the last two decades as projects have grown larger, more complex, and more distributed. Very few real-world systems today are built by one person working alone; teams spread across locations coordinate changes to a shared codebase, review each other's work, track defects, and keep documentation current over the life of a project. Git has become the backbone of this way of working, and Git-based platforms now provide the branching, commit history, and pull-request workflows that make distributed collaboration practical.

Around this core of version control, collaboration platforms have grown to support code review, issue tracking, and documentation as well. At the same time, the rising frequency of security incidents has pushed security review into becoming an essential, if often understaffed, part of the development process. Even with all this tooling in place, most of the supporting work still comes down to manual effort. Reviewing a pull request takes real time and attention; documentation tends to slip when deadlines are tight; catching bugs or vulnerabilities depends heavily on who happens to be reviewing the code and how experienced they are; and any real insight into a project's health or a team's productivity is usually whatever the team manages to piece together by hand.

There is another problem that gets far less attention: consistency. Every repository ends up with its own unwritten rules for naming files, organizing folders, structuring API calls, managing state, and composing components. Standard linters can enforce generic rules handed down from outside the project, but they have no way of learning what a specific team has actually settled into using, and they cannot tell apart file types that are supposed to follow different conventions — a test file, for instance, versus a production page component. Left unchecked, this kind of drift builds up quietly as a codebase grows and as contributors join and leave.

Recent progress in artificial intelligence, particularly large language models and automated code analysis, has opened up ways to reduce both of these burdens. AI techniques can now analyze source code to spot defects, propose improvements, summarize changes, write documentation, and pick out patterns that break from a repository's learned conventions. In most tools available today, though, this kind of intelligence shows up as a separate utility or a third-party plug-in sitting on top of a conventional Git platform, rather than as something built directly into the day-to-day collaboration workflow — which limits how naturally AI assistance can fit into a developer's routine.

The project described in this synopsis, the AI-Powered Dev Collaboration Platform, sets out to close that gap. It combines Git-based version control and team collaboration with a connected set of AI capabilities, including a module built specifically to learn and enforce a repository's own naming, structural, and architectural conventions. Alongside its Git and collaboration foundation, the platform is designed to bring together:

- Repository Management — organizing source code, branches, and version history in one place.
- Team Collaboration — real-time discussion, pull requests, and coordinated teamwork.
- AI Code Review — automated analysis of submitted code for quality, style, and best practice.

- AI Bug Detection — early identification of likely defects and vulnerabilities.
- AI Documentation Generator — automatic API documentation and README generation.
- AI Project Assistant — a conversational assistant for development guidance.
- AI Security Scanner — automated vulnerability detection with recommendations.
- Smart Analytics — data-driven insight into productivity, quality, and trends.
- AI Consistency & Convention Analyzer — learns a repository's naming conventions, groups files into comparable peer groups, and ranks deviations with supporting evidence.

By building these capabilities directly into the Git workflow instead of layering them on afterward, the platform aims to cut down the repetitive manual work around review, documentation, and issue triage, while giving development teams proactive, repository-aware support throughout the software development lifecycle.

LITERATURE SURVEY

This section reviews prior work relevant to each functional area of the proposed platform, organised feature-wise and cited in IEEE style. Each part briefly summarises the relevant research and points out where the proposed system builds on it or departs from it.

3.1 Git-Based Version Control and Collaborative Development

Git is now the de facto standard for distributed version control because of its branching model and its support for decentralized workflows [1]. Studies that have mined Git repositories at scale show that commit and branch histories hold a lot of useful information about how developers behave and how projects evolve, but also warn that this kind of data has to be interpreted carefully to avoid drawing the wrong conclusions [2]. What these studies make clear is that Git itself carries no intelligence about the quality, safety, or consistency of the changes it tracks — that judgment is left entirely to human reviewers or to tools built on top of the platform, which is exactly the gap the proposed AI Intelligence layer is meant to fill.

3.2 Automated and AI-Assisted Code Review

Research into code review coverage shows a clear link between how much reviewers get involved and how many defects actually get caught, which says a lot about how much of software quality still rests on manual review [3]. Bacchelli and Bird's well-known study of code review in practice found that developers value review mainly for catching defects and building a shared understanding of the code, even though the process itself remains slow [4]. A more recent industry study presented at ICSE-SEIP in 2025 looked at AI-assisted review across thousands of real pull requests and reported genuine quality gains alongside some downsides, such as longer times to close a pull request [9] — a reminder that AI-assisted review works best when it is integrated thoughtfully rather than added on top. Large language models trained specifically on code, of the kind evaluated by Chen et al., have already shown a strong ability to reason about program structure and correctness [5], and this body of work forms much of the technical foundation for the proposed AI Code Review module.

3.3 AI-Based Bug Detection and Vulnerability Prediction

A good deal of work has applied machine learning to the problem of finding bugs and vulnerabilities before they ship. Zeng et al.'s survey of deep-learning-based vulnerability detection found that neural approaches can pick up on vulnerable code patterns that purely rule-based analysis tends to miss, though usually at the cost of being harder to interpret [18]. Chakraborty et al. followed up by testing how well these detectors actually generalize outside curated benchmarks, and found that reported accuracy often drops sharply once the data gets messier and more imbalanced [19]. These findings are part of why the proposed AI Bug Detection module is designed to back up its findings with evidence rather than relying on a bare confidence score.

3.4 AI-Assisted Documentation Generation

Automated documentation has moved on from simple comment templates toward generation driven by language models. Khan and Uddin showed that GPT-3-based models can produce working code documentation straight from source, cutting down significantly on the need for developers to write

comments by hand [17]. This work supports the idea behind the proposed AI Documentation Generator, though it also points to an open problem — keeping generated documentation in step with a codebase that keeps changing — which the proposed platform tackles by regenerating documentation on every commit instead of only when someone asks for it.

3.5 AI Project Assistants and Foundational Language-Model Research

Conversational, code-aware assistance owes a lot to the Transformer architecture that Vaswani et al. introduced, which sits underneath almost every large language model in use today [6]. A recent IEEE survey on LLM-based source-code processing points to open challenges such as scaling to large real-world repositories, interpretability, and robustness [10], and a separate systematic review of LLM-based code generation catalogues just how quickly this area has grown, along with the technical gaps that remain [11]. Together, these surveys support treating the proposed AI Project Assistant as a context-aware layer grounded in actual repository evidence, rather than a general-purpose chatbot bolted onto the platform.

3.6 Software Security Analysis

Static and dynamic security analysis are well-established practices, and the OWASP Top Ten remains the industry's go-to reference for the most common vulnerability classes [8]. The deep-learning vulnerability detection research discussed earlier extends this static-analysis tradition with pattern-learning techniques of its own [18][19]. In practice, though, tools of both kinds are usually run as separate pipelines sitting outside the main collaboration platform, which is the reasoning behind building the proposed AI Security Scanner directly into the commit and pull-request workflow rather than running it separately.

3.7 Software Analytics and Productivity Prediction

Meyer et al. studied how developers themselves perceive their own productivity and found that it depends on far more than commit counts — things like progress toward personal goals and how often someone gets interrupted matter just as much [7]. A roundtable of software-analytics researchers convened by IEEE Software later laid out the opportunities and open problems in turning repository and process data into insight that engineering teams can actually act on [20]. Both of these inform the design of the proposed Smart Analytics module, which is meant to surface predictive, explainable signals about productivity, code quality, and project health, rather than just totals and counts.

3.8 Naming Convention Learning and Enforcement

Naming conventions have a real effect on how easy code is to read and maintain. Butler, Wermelinger, and Yu ran a large-scale study of naming-convention adherence across a wide body of Java code and found that the checking tools available at the time could only catch simple, typographic-level conventions, missing the more elaborate patterns that are specific to individual projects [13]. Allamanis, Barr, Bird, and Sutton took this further by proposing that naming conventions should be learned directly from a codebase's own history instead of imposed from an outside style guide, and showed that models trained this way can suggest names for methods and classes that are genuinely consistent with a project's existing conventions [14]. This line of research is the direct basis for the naming-convention sub-feature of the proposed AI Consistency &

Convention Analyzer, which learns whatever naming pattern already dominates a repository — say, PascalCase for component filenames — and flags the files that fall outside it, along with the frequency evidence behind the flag.

3.9 Peer-Group Comparison, Architectural Anomaly and Code-Smell Detection

Detecting architectural anomalies and code smells is an active area of research in its own right. Al Hilmi et al.'s systematic review of sixty-nine primary studies found that machine-learning-based detection has become the dominant approach, though interpretability and dataset bias are still open concerns [15]. More recently, Souza et al. tested large language models for code-smell detection and found that they support a more flexible, context-sensitive style of detection than the fixed thresholds used in classical static analysis [16]. A related strand of work on architecturally relevant code smells argues that anomalies are best judged against a baseline of genuinely comparable components rather than the codebase as a whole, since different parts of a system are often built around different, equally valid design patterns [12]. This peer-group idea — judging a file only against structurally similar "peers," rather than the entire repository at once — is what shapes the proposed platform's peer-group comparison and deviation-ranking features, which score and rank issues by confidence and severity instead of handing out blanket pass/fail results.

3.10 Research Gap

Taken together, this body of work shows that Git-based collaboration, automated code review, defect and vulnerability prediction, documentation generation, security analysis, software analytics, and naming or architectural consistency checking have mostly been studied as separate problems, and that convention-checking tools in particular still lean on generic, externally defined rules instead of conventions actually learned from a project [13][14]. As a result, teams end up stitching together several disconnected tools just to get reasonably complete, repository-aware AI support across the development lifecycle. The AI-Powered Dev Collaboration Platform proposed here aims to close that gap by bringing Git-based collaboration together with seven AI modules — AI Code Review, AI Bug Detection, AI Documentation Generator, AI Project Assistant, AI Security Scanner, Smart Analytics, and the newly added AI Consistency & Convention Analyzer — inside one unified platform.

MOTIVATION

Modern software teams lean heavily on Git-based platforms to coordinate distributed work, but a large share of what it takes to ship good software is still manual and repetitive. Code review remains one of the most time-consuming parts of the development cycle, documentation gets pushed aside whenever deadlines tighten, and bugs or vulnerabilities can sit unnoticed until much later in the process.

There is also a quieter cost that builds up over time: consistency drift. As a repository grows and its contributors change, naming conventions, folder structures, and architectural patterns that used to be followed almost automatically start to pull apart. Ordinary linters cannot really fix this, because they check against generic, predefined rules rather than whatever conventions a team has actually settled into, and they have no way of telling apart file types that are meant to follow different patterns in the first place — test utilities and production components, for example. Left alone, this drift makes onboarding new developers harder, makes automated refactoring riskier, and slowly wears away the very consistency a shared convention was supposed to protect.

These observations are what motivate building a collaboration platform where artificial intelligence is not a separate add-on but a working part of the development process itself — one that reviews code, catches bugs, writes documentation, scans for vulnerabilities, produces analytics, and also learns and protects a repository's own naming and architectural habits through peer-group-aware, evidence-backed deviation detection. The goal behind the AI-Powered Dev Collaboration Platform, then, is to show how weaving AI directly into a collaborative development environment can speed up review turnaround, strengthen documentation practices, catch bugs and vulnerabilities earlier, keep repository-specific consistency intact as teams scale, and generally improve both developer productivity and project transparency.

PROBLEM DEFINITION

a. Problem Statement

Git-based collaboration platforms already do a solid job of handling version control, repository management, and teamwork through commits, branches, and pull requests. What they still leave to developers, though, is a long list of repetitive, time-consuming tasks spread across the development lifecycle — reviewing code, catching bugs and vulnerabilities, writing documentation, running security checks, and making sense of overall project performance. On top of that, these platforms have no built-in way to learn a repository's own naming, structural, or architectural conventions; wherever convention-checking exists at all, it depends on generic external rules that cannot account for a project's own evolved habits or for legitimate differences between groups of files. This dependence on manual effort, combined with the lack of convention awareness, slows down development, raises the odds of missed defects and security gaps, lets consistency drift as a team scales, and limits how much proactive, data-driven insight a team can get into its own project's health.

What is needed, then, is a unified platform that keeps the familiar Git-based collaboration workflow but weaves artificial intelligence directly into it — covering code review, bug detection, documentation, and security, as well as learning and protecting a project's own naming and architectural conventions through peer-group-aware deviation detection.

b. Objectives

1. To build a collaborative platform with solid repository and Git management capabilities.

2. To support real-time team collaboration and communication.

3. To implement AI-powered, automated code review.

4. To use artificial intelligence to flag likely bugs and vulnerabilities.

5. To generate project documentation automatically.

6. To provide an AI project assistant that offers development guidance.

7. To provide AI-based security analysis with actionable recommendations.

8. To surface analytics on project productivity and code quality.

9. To learn a repository's own naming conventions directly from its codebase, rather than from generic external rules.

10. To group structurally comparable files into peer groups, so deviations are judged against a fair, context-aware baseline.

11. To detect, rank, and explain deviations from learned repository conventions, with supporting evidence and confidence scores.

12. To cut down on repetitive manual development activity.

13. To improve overall software development efficiency and quality.

c. Proposed Architecture

The proposed AI-Powered Dev Collaboration Platform follows a layered architecture that connects users to the platform's core Git and collaboration engine, an integrated AI intelligence layer, supporting AI services, persistent storage, and cloud deployment infrastructure. Figure 1 illustrates how these eight layers fit together and how information flows between them.

User Layer: developers, team leads, reviewers, and administrators who interact with the platform day to day.

Frontend Layer: a responsive web interface built with Next.js, React, TypeScript, and Tailwind CSS, covering the dashboard, repository interface, pull requests, issues, code review, analytics, consistency findings, and the AI assistant panel.

Backend / Application Layer: a Node.js and Express.js application layer that exposes a GraphQL API and handles real-time communication through Socket.io and WebSockets.

Git & Collaboration Layer: the core version-control and teamwork engine, covering repository management, branching, commits, pull requests, code review, issue and bug tracking, notifications, and team collaboration.

AI Intelligence Layer: seven dedicated AI modules — AI Code Review, AI Bug Detection, AI Documentation Generator, AI Project Assistant, AI Security Scanner, Smart Analytics, and the newly added AI Consistency & Convention Analyzer, which itself covers naming-convention learning, peer-group comparison, and deviation detection.

AI Services Layer: external AI providers and orchestration tooling — the OpenAI API, Google Gemini API, and LangChain — together with AST-parsing utilities that extract the structural features used for convention learning.

Database & Storage Layer: persistent storage built on PostgreSQL with Prisma ORM, Redis for caching, and Supabase Storage / AWS S3 for objects, including storage for the learned repository convention profiles.

Cloud / Deployment / Monitoring Layer: containerized deployment through Docker and Kubernetes, hosted on Vercel and Railway, and monitored through Sentry, DataDog, and CloudWatch.

Data generally flows top-down — from the user layer through the frontend, backend, Git and collaboration engine, AI intelligence layer, AI services, database and storage, and finally deployment and monitoring. The architecture also supports feedback in the other direction: the Git and Collaboration layer and the AI Intelligence layer exchange information so that AI modules, including the Consistency & Convention Analyzer, can analyze repository activity and return results; the AI Intelligence layer and the Database layer exchange learned convention profiles and AI-generated results; and the Backend layer talks to both the Database and AI Services layers as needed.

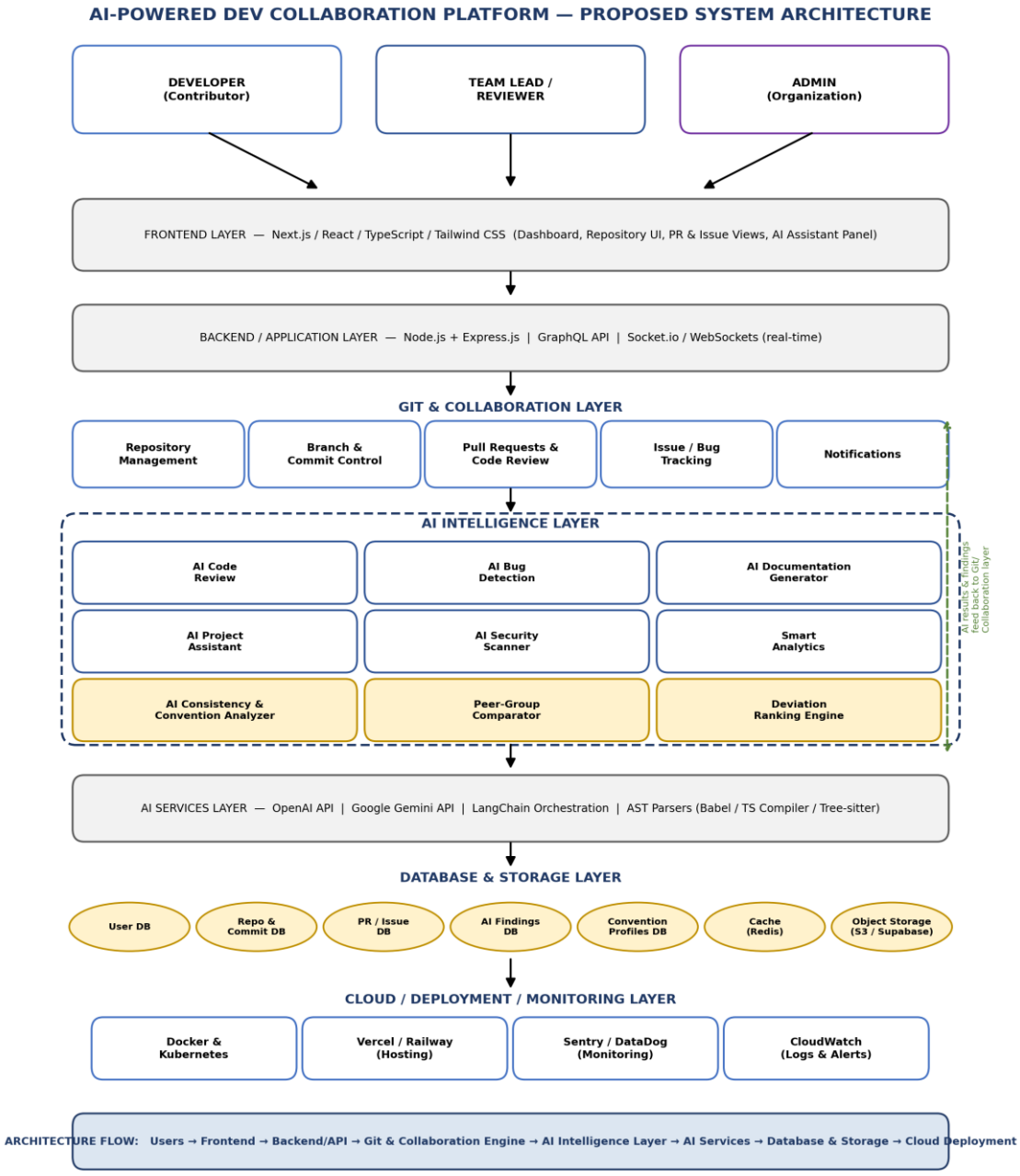


Fig. 1. Proposed system architecture of the AI-Powered Dev Collaboration Platform

Development Workflow

In practice, a typical development cycle moves through this architecture as follows: a developer logs in, selects or initializes a repository, creates a branch, and commits code. Each commit automatically triggers the AI analysis pipeline, where the AI Code Review, Bug Detection, and Security Scanner modules examine the change, the Consistency & Convention Analyzer updates the repository's learned conventions and re-groups affected files into peer groups before flagging any deviations, and the Documentation Generator updates the relevant README and API documentation. These findings are surfaced to the developer and discussed through the pull request, after which Smart Analytics

updates project-level metrics and the reviewed change is merged and deployed through the cloud infrastructure.

d. Experimental Setup

The proposed platform is expected to be built and evaluated using the software and hardware environment summarised below.

Software Requirements

Software Requirement	Details
Operating System	Windows 10/11, Linux, or macOS
Frontend Development	Next.js, React, TypeScript, Tailwind CSS
Backend Development	Node.js, Express.js, GraphQL
Database	PostgreSQL with Prisma ORM
Caching	Redis
AI / ML Services	OpenAI API, Google Gemini API, LangChain
IDE	Visual Studio Code
API Testing	Postman
Version Control	Git & GitHub
Browser	Google Chrome / Microsoft Edge
Diagram Tool	Draw.io / Figma
Containerization & Deployment	Docker, Kubernetes, Vercel, Railway

Hardware Requirements

Hardware Requirement	Minimum Specification
Processor	Intel Core i5 / AMD Ryzen 5 or higher
RAM	8 GB minimum (16 GB recommended)
Storage	20 GB or more available SSD space
Display	1366 × 768 or higher
Internet	Stable broadband connection (required for AI APIs and cloud services)
Server	Cloud-hosted application server (Vercel / Railway)
Backup	Cloud object storage (AWS S3 / Supabase)

Libraries

Frontend

- Next.js & React – web application framework

- TypeScript – type-safe development
- Tailwind CSS – styling and responsive layout
- Recharts – analytics dashboard visualization
- Socket.io-client – real-time updates

Backend

- Express.js – REST / GraphQL API development
- Prisma ORM – database connectivity
- JWT / Clerk / Auth.js – authentication and authorization
- Socket.io – real-time collaboration features

AI / ML

- OpenAI API & Google Gemini API – code review, bug detection, documentation, and assistant capabilities
- LangChain – orchestration of AI calls and prompt chains
- Babel / TypeScript Compiler API / Tree-sitter – AST parsing for convention learning
- Lightweight statistical & embedding models – naming-frequency and peer-group similarity analysis

Database & Services

- PostgreSQL – centralized relational database
- Redis – caching and session management
- Supabase Storage / AWS S3 – object storage
- Docker & Kubernetes – containerized deployment
- Git & GitHub – source-code management

The tools and libraries listed above may be revised during implementation to reflect project requirements and evolving technology choices.

REFERENCES

[1] S. Chacon and B. Straub, Pro Git, 2nd ed. Apress, 2014.

[2] C. Bird, P. C. Rigby, E. T. Barr, D. J. Hamilton, D. M. German, and P. Devanbu, “The promises and perils of mining git,” in Proc. 6th IEEE Int. Working Conf. on Mining Software Repositories (MSR), 2009.

[3] S. McIntosh, Y. Kamei, B. Adams, and A. E. Hassan, “The impact of code review coverage and code review participation on software quality,” in Proc. 11th Working Conf. on Mining Software Repositories (MSR), 2014.

[4] A. Bacchelli and C. Bird, “Expectations, outcomes, and challenges of modern code review,” in Proc. 35th Int. Conf. on Software Engineering (ICSE), 2013.

[5] M. Chen et al., “Evaluating Large Language Models Trained on Code,” arXiv preprint arXiv:2107.03374, 2021.

[6] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention Is All You Need,” in Advances in Neural Information Processing Systems (NeurIPS), 2017.

[7] A. N. Meyer, T. Fritz, G. C. Murphy, and T. Zimmermann, “Software developers' perceptions of productivity,” in Proc. 22nd ACM SIGSOFT Int. Symp. on Foundations of Software Engineering (FSE), 2014.

[8] OWASP Foundation, “OWASP Top Ten,” [Online]. Available: https://owasp.org/www-project-top-ten/.

[9] “Automated Code Review in Practice,” 2025 IEEE/ACM ICSE-SEIP, DOI: 10.1109/ICSE-SEIP66354.2025.00043.

[10] “A Survey and Prospects of Source Code Processing Techniques Based on Large Language Models,” ICSESS 2025, DOI: 10.1109/ICSESS67729.2025.11380277.

[11] “LLM-Based Code Generation: A Systematic Literature Review With Technical and Demographic Insights,” IEEE Access, vol. 13, pp. 194915–194939, 2025.

[12] “Detecting architecturally-relevant code smells in evolving software systems,” background reference for architectural anomaly detection.

[13] S. Butler, M. Wermelinger, and Y. Yu, “Investigating naming convention adherence in Java references,” in Proc. 2015 IEEE 31st Int. Conf. on Software Maintenance and Evolution (ICSME), pp. 41–50, 2015. DOI: 10.1109/ICSM.2015.7332450.

[14] M. Allamanis, E. T. Barr, C. Bird, and C. Sutton, “Learning natural coding conventions,” in Proc. 22nd ACM SIGSOFT Int. Symp. on Foundations of Software Engineering (FSE), pp. 281–293, 2014.

[15] M. A. Al Hilmi, A. Puspaningrum, Darsih, D. O. Siahaan, H. S. Samosir, and A. S. Rahma, “Research Trends, Detection Methods, Practices, and Challenges in Code Smell: SLR,” IEEE Access, vol. 11, pp. 129536–129551, 2023. DOI: 10.1109/ACCESS.2023.3334258.

[16] S. Souza, A. Santana, E. Figueiredo, I. Muzetti, J. E. Montandon, and L. Briand, “Beyond Strict Rules: Assessing the Effectiveness of Large Language Models for Code Smell Detection,” IEEE Transactions on Software Engineering, 2025.

[17] J. Y. Khan and G. Uddin, “Automatic code documentation generation using GPT-3,” in Proc. 37th IEEE/ACM Int. Conf. on Automated Software Engineering (ASE), pp. 1–6, 2022.

[18] P. Zeng, G. Lin, L. Pan, Y. Tai, and J. Zhang, “Software Vulnerability Analysis and Discovery Using Deep Learning Techniques: A Survey,” IEEE Access, vol. 8, pp. 197158–197172, 2020. DOI: 10.1109/ACCESS.2020.3034766.

[19] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, “Deep Learning based Vulnerability Detection: Are We There Yet?,” IEEE Transactions on Software Engineering, 2021.

[20] A. E. Hassan, A. Hindle, P. Runeson, M. Shepperd, P. T. Devanbu, and S. Kim, “Roundtable: What's Next in Software Analytics,” IEEE Software, vol. 30, no. 4, 2013.

APPROVAL STATEMENT

This is to certify that the synopsis report entitled “AI-Powered Dev Collaboration Platform” has been thoroughly reviewed by the expert panel members of the Department of CSE (Data Science), D. Y. Patil College of Engineering & Technology, Kolhapur. The content of the synopsis report has been verified for accuracy, and the proposed work has been approved for further development.

Roll No.	Unique ID	Student's Name	Signature
109	[UNIQUE ID]	Sneha Suresh Abdar	
110	[UNIQUE ID]	Madhura Sandip Patil	
111	[UNIQUE ID]	Saksham Madhukar Wayadande	
112	[UNIQUE ID]	Priyanka Pramod Jadhav	

Date: _____

Place: Kolhapur

Project Guide	Project Coordinator	HOD (CSE Data Science)
Prof. Tejashri Deokar	[Project Coordinator Name]	[HOD Name]

PLAGIARISM REPORT

Attach the screenshot of the plagiarism / similarity report generated from the institute's authorized plagiarism-detection system (e.g., Turnitin / iThenticate), confirming an overall similarity below the permissible threshold.

[INSERT PLAGIARISM REPORT SCREENSHOT HERE]